

Day 10 CS570

Encoders and Gyros

Encoders

Encoders are a way to find out how much a motor has moved. There are many different kind of encoders, but we will focus on the kind of encoders that are on the ROMI's. These are **quadrature** encoders. They can tell you how far the wheels have moved by counting ticks of 2 sensors. Watch this youtube video to see more about how this works:

<https://www.youtube.com/watch?v=p4BCFhIuC88>

or more information can be found at:

<https://www.youtube.com/watch?v=oLBYHbLO8W0>

On the ROMI the encoders are visible. They are the little black pieces on the ends of the white motors that are attached to the main board.



The math of encoders The ROMI's encoders measure how much the motor has turned, but the motor turns the a gearbox which turns the wheel, and the size of the wheel contributes to how far the ROMI will move.

Challenge Work with other students near you to determine how much the ROMI moves in meters for one tick of the encoder.

You will need more information about the ROMI's motors, encoders, and gearing. I suggest that you look at this website: [Getting to know your ROMI](#)

Coding for Encoders

Drivetrain file For all the following work we do with the ROMI it will be good to have a **Drivetrain** class that we can keep the code for how the robot drives with in one place. Make a folder in your project called **subsystems**, and in there put a file called **drivetrain.py**. The file should include the following code.

Drivetrain The drivetrain code is as follows:

```
import math

import wpilib
import wpilib.drive
import romi

class Drivetrain():
    kCountsPerRevolution = 1440.0
    kWheelDiameterInch = 2.75591

    def __init__(self) -> None:
        super().__init__()

        # The Romi has the left and right motors set to
        # PWM channels 0 and 1 respectively
        self.leftMotor = wpilib.Spark(0)
        self.rightMotor = wpilib.Spark(1)

        # The Romi has onboard encoders that are hardcoded
        # to use DIO pins 4/5 and 6/7 for the left and right
        self.leftEncoder = wpilib.Encoder(4, 5)
        self.rightEncoder = wpilib.Encoder(6, 7)

        # Set up the differential drive controller
        self.drive = wpilib.drive.DifferentialDrive(self.leftMotor, self.rightMotor)

        # Set up the RomiGyro
        self.gyro = romi.RomiGyro()

        # Set up the BuiltInAccelerometer
        self.accelerometer = wpilib.BuiltInAccelerometer()

        # Use inches as unit for encoder distances
        self.leftEncoder.setDistancePerPulse(
            (math.pi * self.kWheelDiameterInch) / self.kCountsPerRevolution
        )
        self.rightEncoder.setDistancePerPulse(
            (math.pi * self.kWheelDiameterInch) / self.kCountsPerRevolution
        )
        self.resetEncoders()

    def arcadeDrive(self, fwd: float, rot: float) -> None:
        """
        Drives the robot using arcade controls.

        :param fwd: the commanded forward movement
        :param rot: the commanded rotation
        """
        self.drive.arcadeDrive(rot, fwd)

    def resetEncoders(self) -> None:
        """Resets the drive encoders to currently read a position of 0."""
        self.leftEncoder.reset()
        self.rightEncoder.reset()
```

```

def getLeftEncoderCount(self) -> int:
    return self.leftEncoder.get()

def getRightEncoderCount(self) -> int:
    return self.rightEncoder.get()

def getLeftDistanceInch(self) -> float:
    return self.leftEncoder.getDistance()

def getRightDistanceInch(self) -> float:
    return self.rightEncoder.getDistance()

def getAverageDistanceInch(self) -> float:
    """Gets the average distance of the TWO encoders."""
    return (self.getLeftDistanceInch() + self.getRightDistanceInch()) / 2.0

def getAccelX(self) -> float:
    """The acceleration in the X-axis.

    :returns: The acceleration of the Romi along the X-axis in Gs
    """
    return self.accelerometer.getX()

def getAccelY(self) -> float:
    """The acceleration in the Y-axis.

    :returns: The acceleration of the Romi along the Y-axis in Gs
    """
    return self.accelerometer.getY()

def getAccelZ(self) -> float:
    """The acceleration in the Z-axis.

    :returns: The acceleration of the Romi along the Z-axis in Gs
    """
    return self.accelerometer.getZ()

def getGyroAngleX(self) -> float:
    """Current angle of the Romi around the X-axis.

    :returns: The current angle of the Romi in degrees
    """
    return self.gyro.getAngleX()

def getGyroAngleY(self) -> float:
    """Current angle of the Romi around the Y-axis.

    :returns: The current angle of the Romi in degrees
    """
    return self.gyro.getAngleY()

def getGyroAngleZ(self) -> float:
    """Current angle of the Romi around the Z-axis.
    This is the angle you would want for the turn in 2D space
    :returns: The current angle of the Romi in degrees
    """

```

```

        return self.gyro.getAngleZ()

def resetGyro(self) -> None:
    """Reset the gyro"""
    self.gyro.reset()

```

There are Encoder objects in wpilib:

```

from wpilib import Spark, Encoder

...

self.leftEncoder = Encoder(4, 5)
self.rightEncoder = Encoder(6, 7)

```

The encoders are attached to pins 4 and 5 on the left and 6 and 7 on the right on the ROMI. The ROMI's take two pins because they are quadrature encoders that allow us to determine direction as well as distance.

You can let the encoders know how much each tick is worth in terms of distance using the `setDistancePerPulse` method.

You can reset the encoders back to zero using the `reset` method.

You can get the number of ticks the encoder has registered using the `get` method.

If you set the conversion for ticks to distance then you can use the encoder's `getDistance` method to find the distance travelled.

Project 3: Make ROMI go 5 feet in a straight line With the information above let's think about how we would make an autonomous routine where the ROMI moves straight for 2 meters.

Project Requirements The robot must be driveable in Teleop

- When you exit Autonomous you must make it so that the robot will perform the autonomous again when you reenter Autonomous.

Drivetrain class

- You must have a drivetrain class, that you use to drive the robot in Teleop and that you pass to your DriveStraight class.
- the drivetrain class must implement the motor encoders, and you must have methods that allow you to determine how far each wheel has rotated, and how much distance that would have covered in total.
- You can have other methods that you find useful as well, you may want to have a method that averages the distances the wheels have traveled for instance.

DriveStraight Class

This class will be in charge of the robot during autonomous.

- The DriveStraight class should take in a drivetrain and how far it is supposed to drive straight in meters.
- The DriveStraight class should have a run method that gets called in `autonomousPeriodic`. In the run method, the class should determine speed and direction based on information from the motor encoders.

Warning

- The motors will not run straight exactly even if you tell them to. You will need to turn the robot to compensate.
- You may have to tune your constants a little to get the robot to drive the right distance.